

复旦大学

本科生课程实验报告

实验题目:	Lab0: GitLab
修读课程:	计算机系统基础
姓 名:	雍畅
学 号:	25800680018
完成日期:	2026年9月30日

摘要

本实验于2026年9月29日在Windows 11与WSL 2 Ubuntu环境中完成。我基于课程模板建立个人仓库，完成main.c修改、暂存与提交，并在main和feature分支分别修改同一条打印语句，通过处理合并冲突保留双方输出。

基线提交C为2766614，main与feature分别形成e2c5045和49e68ed。两条开发线改变了共同起点的同一条打印语句，合并时产生内容冲突。保留两条输出后建立合并提交ec5e612，其两个父节点按顺序为main端点和feature端点。新目录重新克隆得到相同提交，并成功输出两行问候语。

合并版本对应的GitHub Actions自动测试得分为100/100，新克隆目录也成功构建并输出两行问候语。通过本次实验，我理解了暂存区对提交内容的控制、分支的独立发展，以及合并提交对双方历史的记录。

关键词：Git；暂存区；分支；合并冲突；GitHub Actions

目录

一、文档问题回答	1
1. 协同开发经历	1
2. 暂存与提交两个步骤	2
3. 本地分支与全部分支查询	3
二、个人仓库与首次提交	4
1. 实验环境	4
2. 使用模板建仓与克隆	7
3. 修改与暂存代码	8
4. 首次提交与运行	10
三、拓展阅读与学习认识	11
1. Commit Message规范	11
2. Gitflow分支组织	11
3. 为什么学习Git	11
四、分支管理与冲突处理	14
1. 创建feature分支	14
2. 分别修改两个分支	14
3. 比较合并前的版本	17
4. 触发并解决合并冲突	17
5. 建立合并提交	20
6. 推送与自动测试	20
7. 重新克隆并运行	22
五、实验结果与总结	24

一、文档问题回答

1.协同开发经历

在此前的实习中，我曾与另外两名实习生及mentor协作完成ETF研究和数据处理任务。项目中，我们一方面分工完成ETF相关资料整理与研究报告撰写，最终形成完整的研究报告；另一方面共同完成数据爬取、清洗、字段整理和汇总检查，形成了一套能够衔接运行的Python数据处理代码及相应的数据结果。整个任务涉及多人任务分工、文件交换、代码修改、版本整合和整体运行检查。

在任务开始阶段，我们首先按照工作内容进行分工。三名实习生分别负责部分数据采集、清洗和字段整理工作，mentor负责整体要求、数据口径以及最终汇总检查。由于不同代码模块之间存在前后依赖关系，采集结果需要作为清洗模块的输入，清洗后的数据还需要继续进入汇总代码，因此每个人在完成自己的代码同时，还需要保持各模块之间有一致的数据接口。

实际协作中，我们曾遇到字段名称、日期格式、缺失值表示和公共函数参数不一致的问题。例如，前一个模块能够正常输出数据，但由于字段格式发生变化，后一个模块可能无法按照原有方式读取；公共函数参数修改后，如果其他成员仍按照旧接口调用，也会造成程序无法运行。为减少这类问题，我们统一了字段名称、数据类型、日期格式和输出目录，并在交付代码时说明修改内容、输入输出变化以及可能影响的其他模块。

当时我们主要通过文件备份、版本编号和修改说明管理代码，尚未使用Git。进行较大修改前，我会保留当前能够正常运行的版本，并按照日期和连续编号保存不同阶段的文件；每轮交付时，则记录负责人、文件版本、主要修改内容和相关依赖。对于已经完成整体检查的程序，我们会保留一份完整代码快照，作为后续修改的基础版本。

这种方式能够实现基本的历史恢复，但随着文件数量增加，也暴露出一些局限。单纯依靠“新版”“最终版”或日期命名，很难准确判断某个文件包含了哪些修改；而且不同模块各自的最新版本并不一定能够相互兼容。如果直接将每个人最后修改的文件放在一起，仍可能出现接口不一致或依赖关系错误。因此，在实际协作中，我们除了记录文件版本外，还需要同时记录主要修改内容及相关模块之间的依赖关系。

代码汇总阶段，我们会根据修改说明比较不同文件之间的变化，并由固定成员完成整合。对于公共函数、数据结构和处理流程的修改，需要特别确认是否影响其他模块。完成代码合并后，我们使用同一批样本运行从数据采集、数据处理到最终

结果输出的完整流程，检查字段是否正确、模块是否能够衔接，以及输出结果是否符合要求。

如果整体运行出现问题，我们会根据备份版本和修改记录比较前后变化，定位可能受到影响的代码，再调整相关模块并重新运行完整流程。最终保留一套能够相互配合并通过整体检查的完整代码。

此外，在ETF研究报告的协作中，我们还使用了Word的批注、修订和版本备份功能。mentor会通过批注提出数据口径、研究逻辑和文字表述方面的修改意见，各成员根据负责范围修改相应内容并保留修订记录。较大调整前还会保存带有日期和版本编号的阶段性文件，由固定成员负责汇总不同成员的修改。对于同一部分存在多份修改的情况，则通过Word的比较、合并功能配合人工检查完成整合。

总体而言，这段经历让我对计算机多人协同有了更加具体的认识。多人协作不仅需要进行任务分工，还需要统一模块接口和数据格式，并对修改内容、版本状态和依赖关系进行记录。代码整合也不能停留在文件层面的合并，还需要通过整体运行检查确认不同模块能够正确衔接。

结合本次实验，我能够把当时的阶段性快照理解为可运行基线，把不同成员的修改理解为从共同起点发展的开发线。Git可以明确保存这些关系，而接口口径与整体验证仍由协作成员共同完成。

2.暂存与提交两个步骤

暂存与提交分离，使开发者能够在形成历史版本前选择并检查本次内容。同一工作区可能同时包含已完成和仍在调试的修改；先add再commit，可以让一次提交围绕明确任务组织。普通git commit使用暂存区内容，因此工作区中的其他变化不会自动进入该提交。

工作区保存可编辑文件，暂存区保存下一次提交准备记录的内容，commit把该快照写入历史。git add读取当时磁盘中的文件；执行后继续编辑会产生新的未暂存变化。

$$W \rightarrow I \rightarrow H$$

W表示工作区，I表示暂存区，H表示提交历史；两次箭头分别对应git add和git commit。git diff -- main.c比较工作区与暂存区，本实验用它查看尚未暂存的修改。git diff --cached -- main.c比较暂存区与HEAD，用于检查准备写入提交的内容。git status显示文件状态和合并状态，用于区分未暂存、已暂存和正在合并的状态。

本实验图11与图12给出了直接证据：main.c的修改起初列在未暂存区域，add后进入待提交区域；此时针对main.c的普通diff为空，而--cached仍显示修改。提交C后，

git status显示工作区干净，表示当时没有需由Git报告的已暂存或未暂存修改，也未显示未跟踪文件；提交内容仍需结合差异和提交记录核对。

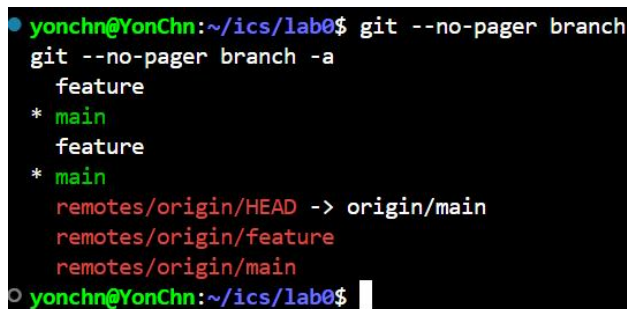
我把暂存区理解为提交内容的选择与核对环节。在后续实验中，完成一个独立目标后，先检查差异，再暂存相关文件，最后核对--cached输出并提交，可以让版本记录更便于阅读、比较和问题定位。

3.本地分支与全部分支查询

```
git --no-pager branch
```

```
git --no-pager branch -a
```

本实验的本地分支列表显示feature和* main，说明本地存在feature、main两条分支，其中星号表示当前位于main。远程跟踪分支列表显示remotes/origin/main和remotes/origin/feature，它们表示本地保存的origin分支状态。列表中的remotes/origin/HEAD -> origin/main是本地保存的远程HEAD符号引用，指向origin/main；它与本次实验记录中的远程默认分支main相对应。



```
yonchn@YonChn:~/ics/lab0$ git --no-pager branch
git --no-pager branch -a
feature
* main
feature
* main
remotes/origin/HEAD -> origin/main
remotes/origin/feature
remotes/origin/main
yonchn@YonChn:~/ics/lab0$
```

图1 本地分支和远程跟踪分支列表

git branch主要回答本地维护哪些开发线；-a还列出远程跟踪引用。它们保存的是本地已取得的远程状态，并非命令执行时实时查询服务器。本次先推送再fetch后查询，因此origin/main和origin/feature与前述提交图对应。

对多人协作，这一区分有助于识别自己正在修改的本地分支和已经取得的共享分支。远程跟踪分支反映本地上次取得的相应远程开发线状态；判断某项工作是否完成，还要查看提交内容与测试结果。

二、个人仓库与首次提交

1.实验环境

项目	实际信息
Windows	Windows 11家庭中文版；25H2；内部版本26200.9457
WSL与Linux	WSL 2；Ubuntu-22.04；Ubuntu 22.04.5 LTS
架构	x86_64
Git	2.34.1
GCC	11.4.0；Ubuntu 11.4.0-1ubuntu1~22.04.3
Make	GNU Make 4.3
OpenSSH	8.9p1 Ubuntu-3ubuntu0.17
VS Code	通过WSL连接Ubuntu-22.04；报告整理时本机安装版本为1.139.1
实验目录	/home/yonchn/ics/lab0
复现目录	/home/yonchn/ics/lab0-check

Windows提供桌面界面，WSL中的Ubuntu执行Git、编译和运行操作，VS Code通过WSL连接实验目录。



图2 Windows版本信息

```
C:\Users\YonChn>wsl --list --verbose
  NAME                STATE          VERSION
*  Ubuntu              Stopped        2
   Ubuntu-22.04        Running         2

C:\Users\YonChn>
```

图3 Ubuntu发行版采用WSL 2

```
2026-09-29T10:31:39+08:00
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
x86_64
git version 2.34.1
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

GNU Make 4.3
Built for x86_64-pc-linux-gnu
Copyright (C) 1988-2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
OpenSSH_8.9p1 Ubuntu-3ubuntu0.17, OpenSSL 3.0.2 15 Mar 2022
/home/yonchn/ics
yonchn@YonChn:~/ics$
```

图4 Ubuntu与Git GCC Make版本

原sources.list使用Ubuntu官方归档地址。软件包索引更新后，jammy、jammy-updates 和 jammy-backports 使用 mirrors.ustc.edu.cn ， jammy-security 保留 security.ubuntu.com。

```

Fetches 50.4 MB in 22s (2293 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
151 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

图5 软件包索引更新结果

```

yonchn@YonChn:~/ics/lab0$ cat /etc/apt/sources.list
# See http://help.ubuntu.com/community/UpgradeNotes for how to upgrade to
# newer versions of the distribution.
deb http://mirrors.ustc.edu.cn/ubuntu/ jammy main restricted
# deb-src http://mirrors.ustc.edu.cn/ubuntu/ jammy main restricted

## Major bug fix updates produced after the final release of the
## distribution.
deb http://mirrors.ustc.edu.cn/ubuntu/ jammy-updates main restricted
# deb-src http://mirrors.ustc.edu.cn/ubuntu/ jammy-updates main restricted

## N.B. software from this repository is ENTIRELY UNSUPPORTED by the Ubuntu
## team. Also, please note that software in universe WILL NOT receive any
## review or updates from the Ubuntu security team.
deb http://mirrors.ustc.edu.cn/ubuntu/ jammy universe
# deb-src http://mirrors.ustc.edu.cn/ubuntu/ jammy universe
deb http://mirrors.ustc.edu.cn/ubuntu/ jammy-updates universe
# deb-src http://mirrors.ustc.edu.cn/ubuntu/ jammy-updates universe

## N.B. software from this repository is ENTIRELY UNSUPPORTED by the Ubuntu
## team, and may not be under a free licence. Please satisfy yourself as to
## your rights to use the software. Also, please note that software in
## multiverse WILL NOT receive any review or updates from the Ubuntu
## security team.
deb http://mirrors.ustc.edu.cn/ubuntu/ jammy multiverse
# deb-src http://mirrors.ustc.edu.cn/ubuntu/ jammy multiverse
deb http://mirrors.ustc.edu.cn/ubuntu/ jammy-updates multiverse
# deb-src http://mirrors.ustc.edu.cn/ubuntu/ jammy-updates multiverse

## N.B. software from this repository may not have been tested as
## extensively as that contained in the main release, although it includes
## newer versions of some applications which may provide useful features.
## Also, please note that software in backports WILL NOT receive any review
## or updates from the Ubuntu security team.
deb http://mirrors.ustc.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
# deb-src http://mirrors.ustc.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse

deb http://security.ubuntu.com/ubuntu/ jammy-security main restricted
# deb-src http://security.ubuntu.com/ubuntu/ jammy-security main restricted
deb http://security.ubuntu.com/ubuntu/ jammy-security universe
# deb-src http://security.ubuntu.com/ubuntu/ jammy-security universe
deb http://security.ubuntu.com/ubuntu/ jammy-security multiverse
# deb-src http://security.ubuntu.com/ubuntu/ jammy-security multiverse

```

图6 实验后核对的软件源配置

本地Git作者名为YonChn，邮箱为267317446+KlineCY@users.noreply.github.com，配置来源为/home/yonchn/.gitconfig。GitHub账号为KlineCY。作者信息用于提交元数据；SSH认证用于连接远程仓库，两者分别核对。

```

git config --show-origin --get user.name
git config --show-origin --get user.email
ssh -T git@github.com

```

SSH返回Hi KlineCY!及认证成功提示，同时说明GitHub不提供shell访问。这表明密钥身份认证成功，符合使用Git传输的用途。

```

yonchn@YonChn:~/ics$ ssh -T git@github.com
The authenticity of host 'github.com (20.205.243.166)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
Enter passphrase for key '/home/yonchn/.ssh/id_ed25519':
Hi KlineCY! You've successfully authenticated, but GitHub does not provide shell access.

```

图7 GitHub SSH认证成功

2.使用模板建仓与克隆

在课程模板仓库中选择Use this template，再使用Create a new repository创建个人仓库。表单中owner为KlineCY，仓库名为ics-lab0，可见性为Public，未开启Include all branches。创建后网页显示Generated from ICS-26Fall-FDU/GitLab。

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

Start with a template

Templates pre-configure your repository with files.

ICS-26Fall-FDU/GitLab

Include all branches

If enabled, all branches from the template repository will be included.

Off

1 General

Owner *

KlineCY

Repository name *

ics-lab0

ics-lab0 is available.

Great repository names are short and memorable. How about [ideal-winner](#)?

Description

ICS Lab0 Git experiment

23 / 350 characters

2 Configuration

Choose visibility *

Public

Choose who can see and commit to this repository

Create repository

图8 模板建仓表单

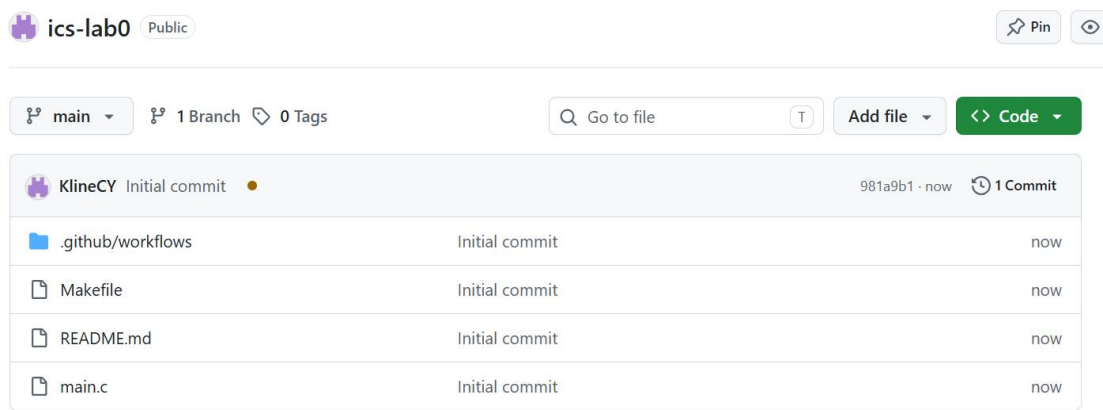


图9 个人仓库初始文件

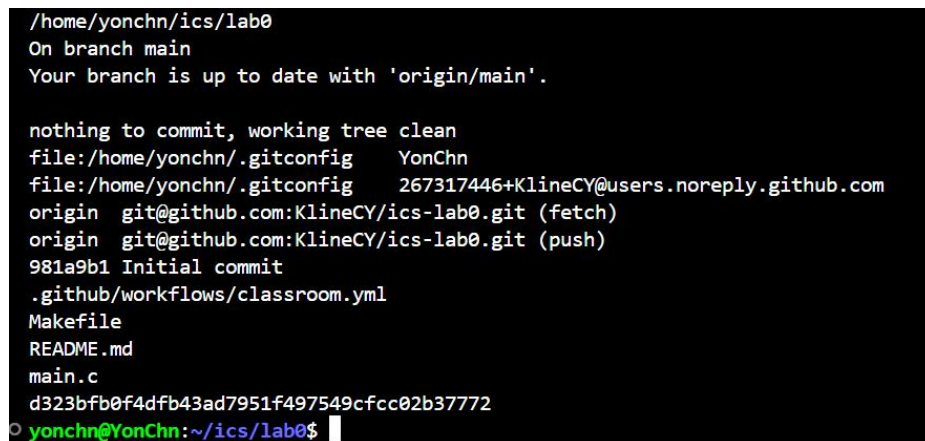
在~/ics目录下克隆个人仓库，再进入lab0工作目录。

```

git clone git@github.com:KlineCY/ics-lab0.git lab0
cd ~/ics/lab0
pwd
git status
git remote -v
git --no-pager log -1 --oneline
git ls-files

```

克隆后的路径为/home/yonchn/ics/lab0，当前分支为main。初始提交为981a9b1，工作区干净，本地分支与origin/main同步。origin的fetch与push地址均为git@github.com:KlineCY/ics-lab0.git。



```

/home/yonchn/ics/lab0
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
file:/home/yonchn/.gitconfig  YonChn
file:/home/yonchn/.gitconfig  267317446+KlineCY@users.noreply.github.com
origin  git@github.com:KlineCY/ics-lab0.git (fetch)
origin  git@github.com:KlineCY/ics-lab0.git (push)
981a9b1 Initial commit
.github/workflows/classroom.yml
Makefile
README.md
main.c
d323bfb0f4dfb43ad7951f497549cfcc02b37772
yonchn@YonChn:~/ics/lab0$

```

图10本地初始状态和远程地址

3.修改与暂存代码

我将模板中的Hello, world!输出改为ICS Lab0: baseline message.，删除TODO注释，并使用int main(void)与return 0。该提交建立后续分支实验的共同起点C。

```

#include <stdio.h>

int main(void)
{
    printf("ICS Lab0: baseline message.\n");
    return 0;
}

git status
git --no-pager diff -- main.c
git add main.c
git status
git --no-pager diff -- main.c
git --no-pager diff --cached -- main.c

```

暂存前，git status将main.c列在Changes not staged for commit下，针对main.c的普通diff显示函数声明与打印语句变化，并新增return 0。执行git add main.c后，状态变为Changes to be committed；针对main.c的普通diff没有输出，--cached显示同一组修

改，说明就main.c而言，工作区版本与暂存区版本一致，而暂存区版本仍不同于HEAD。

```
yonchn@YonChn:~/ics/lab0$ git status
git --no-pager diff -- main.c
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   main.c

no changes added to commit (use "git add" and/or "git commit -a")
diff --git a/main.c b/main.c
index 2122766..2fcf8c9 100644
--- a/main.c
+++ b/main.c
@@ -1,7 +1,7 @@
#include <stdio.h>

-int main()
+int main(void)
{
-    // @TODO: print a sentence you want.
-    printf("Hello, world!\n");
+    printf("ICS Lab0: baseline message.\n");
+    return 0;
}
\ No newline at end of file
yonchn@YonChn:~/ics/lab0$
```

图11 首次修改的未暂存差异

```
yonchn@YonChn:~/ics/lab0$ git add main.c
git status
git --no-pager diff -- main.c
git --no-pager diff --cached -- main.c \
| tee ~/ics/lab0-evidence/E04-staged.diff
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   main.c

diff --git a/main.c b/main.c
index 2122766..2fcf8c9 100644
--- a/main.c
+++ b/main.c
@@ -1,7 +1,7 @@
#include <stdio.h>

-int main()
+int main(void)
{
-    // @TODO: print a sentence you want.
-    printf("Hello, world!\n");
+    printf("ICS Lab0: baseline message.\n");
+    return 0;
}
\ No newline at end of file
yonchn@YonChn:~/ics/lab0$
```

图12 首次修改进入暂存区

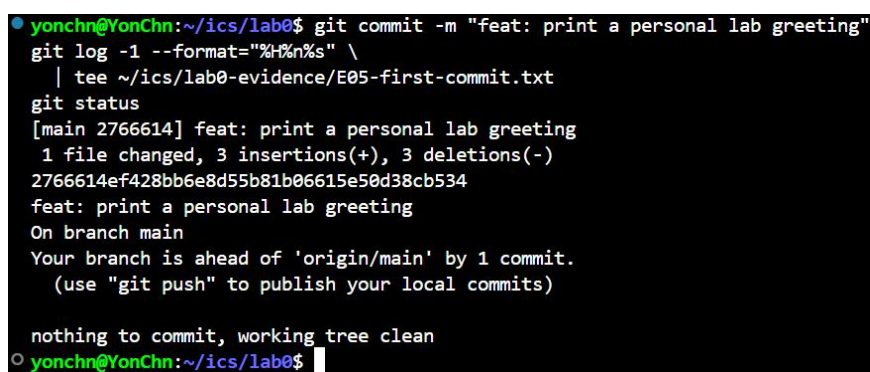
4.首次提交与运行

```
make
./main
git commit -m "feat: print a personal lab greeting"
git log -1 --format="%H%n%s"
git status

git push origin main
```

首次修改保存后，编译运行得到基线输出。提交C的七位短哈希为2766614；状态检查时工作区干净，未显示未跟踪的编译产物，本地main比origin/main领先1个提交。首次推送将该基线同步到远程。

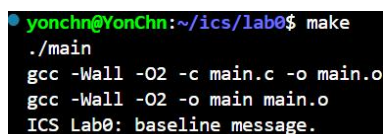
ICS Lab0: baseline message.

A terminal window showing the execution of git commands. The user runs 'git commit -m "feat: print a personal lab greeting"', followed by 'git log -1 --format="%H%n%s" \ | tee ~/ics/lab0-evidence/E05-first-commit.txt', and then 'git status'. The status output shows a new commit on the main branch with 1 file changed and 3 insertions/deletions. It also indicates the branch is ahead of origin/main by 1 commit.

```
yonchn@YonChn:~/ics/lab0$ git commit -m "feat: print a personal lab greeting"
git log -1 --format="%H%n%s" \
| tee ~/ics/lab0-evidence/E05-first-commit.txt
git status
[main 2766614] feat: print a personal lab greeting
 1 file changed, 3 insertions(+), 3 deletions(-)
2766614ef428bb6e8d55b81b06615e50d38cb534
feat: print a personal lab greeting
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
yonchn@YonChn:~/ics/lab0$
```

图13 基线提交与工作区状态

A terminal window showing the execution of the 'make' command. The output shows the compilation of main.c to main.o and the execution of the program, which prints 'ICS Lab0: baseline message.'.

```
yonchn@YonChn:~/ics/lab0$ make
./main
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
ICS Lab0: baseline message.
```

图14 基线程序运行结果

提交C记录了当时Git跟踪内容的项目快照；运行输出说明该版本在所记录环境中能够构建并执行。两类证据共同建立分支实验的基线。

三、拓展阅读与学习认识

1. Commit Message规范

《Commit message和Change log编写指南》主要介绍了commit message的规范写法及其在版本管理中的作用。文章指出，每次进行commit时，都应当通过commit message说明本次修改的内容和目的，使项目的版本历史更加清晰，也便于之后查看不同阶段的修改记录。

文章以Angular的commit message规范为例，将提交信息划分为Header、Body和Footer三个部分。其中，Header通常包含type、scope和subject。type用于表示修改类别，例如feat表示新增功能，fix表示修复错误，docs表示文档修改，refactor表示代码重构；scope用于说明修改影响的范围；subject则用于简要概括本次修改。Body可以进一步说明修改原因和具体变化，Footer可以记录不兼容修改或关联的Issue。

规范的commit message还可以用于筛选不同类型的commit，并进一步整理Change log。由此，commit message可以为每次修改补充较为统一的信息，使之后阅读和追踪项目历史更加方便。

2. Gitflow分支组织

《Gitflow使用规范》主要介绍了如何利用不同分支组织开发任务。文章将master、develop、feature、release和hotfix等分支赋予不同职责：master用于保存正式发布的稳定版本，develop用于日常开发，新功能可以在feature分支中独立完成，release和hotfix则分别对应发布准备和紧急修复。

Gitflow的核心思路是让不同性质的工作先在相对独立的分支中进行，再按照既定关系进行合并。这样可以使不同任务具有较清楚的来源和去向，也便于多人同时推进不同工作。文章同时介绍了创建、切换和合并分支等基本操作，使分支管理与实际开发流程相对应。

对于目前接触的课程项目和简单多人协作，并不一定需要完整采用所有Gitflow分支，其中通过分支区分任务、完成后再进行整合的基本思路具有较强的参考意义。

3. 为什么学习Git

在此前的编程学习和实习协作中，已经接触过不少与版本管理有关的问题。第一部分中的协同开发经历表明，随着修改次数增加和参与人数增多，依靠文件备份、版本编号和修改说明，需要投入越来越多的精力维护不同版本之间的关系。Git能够把代码在编写、修改、测试和整合过程中产生的变化组织到统一的版本历史中，使代码从一个状态发展到另一个状态的过程更加清楚。

结合前两篇拓展阅读来看，Git的价值主要体现在两个方面：一方面，通过commit和commit message记录代码随时间发生的变化；另一方面，通过分支组织不同成员和不同任务之间的并行开发。这两种机制分别对应版本历史和多人协作，也构成了学习Git最直接的现实基础。

3.1 版本记录与问题定位

个人编程中，常见的做法是通过多个文件保存不同阶段的代码。这种方式能够保留历史版本，但随着修改次数增加，很难仅凭文件名判断每一版具体发生了什么变化。Git中的commit可以将一次相对完整的修改保存为历史节点，多个commit按照实际修改顺序形成连续的版本记录，commit message则进一步说明每次修改的内容和目的。

这种记录方式也为之后的代码检查和问题定位提供了依据。通过git log可以查看已有的commit及其先后顺序，通过git diff <commit1> <commit2>可以比较两个版本之间的代码差异。当程序在连续修改后出现报错或异常结果时，可以沿着版本历史查看近期变化，再根据差异逐步缩小检查范围。

版本历史还能够支持代码恢复。Git会保存各个commit对应的历史状态，因此可以查看此前能够正常运行的版本。如果希望放弃某个文件尚未暂存的修改，可以使用git restore 文件名恢复工作区中的文件；如果需要恢复指定commit中的文件，则可以使用git restore --source=<commit> -- 文件名。对于课程实验和日常编程中经常出现的试验性修改，这种方式能够保留已经验证过的代码，也使后续调整和恢复具有明确的版本依据。

由此，版本管理的关注对象逐渐从保存若干阶段性文件扩展到记录代码变化本身。一次修改从哪个状态开始、具体改变了什么以及之后是否继续保留，都可以沿着版本历史进行追踪，使代码的修改、检查和恢复形成连续的过程。

3.2 多人协作与分支组织

多人协作中，不同成员的工作进度和修改内容往往并不完全同步。Git通过分支将不同任务放在相对独立的开发线上，使各成员能够从共同版本出发完成各自的修改，再将确认后的内容进行整合。这与Gitflow中通过不同分支组织开发任务的思路相一致。

分支机制能够使不同修改具有较明确的来源和去向，但代码最终能否正常运行，还需要配合相应的协作规范。不同模块之间的字段名称、数据格式、输入输出和公共函数接口仍然需要保持一致；在合并前需要确认重要接口是否发生变化，在合并后还需要通过整体运行检查不同模块能否正确衔接。

因此，Git中的分支和合并主要负责组织不同成员和不同任务之间的修改关系，接口约定和整体测试则进一步保证这些修改能够在实际程序中共同运行。对于课程小组项目和多人数据处理任务，这种方式可以使分工、修改和最终整合之间形成更加清楚的联系。

3.3 远程协作与实际应用

Git的使用范围还延伸到了远程代码托管和开源协作。GitHub、Gitee等平台都以Git仓库为基础，本地代码和版本历史可以通过远程仓库在不同计算机和不同成员之间共享。实际使用时，可以通过`git clone`获取远程仓库，通过`git pull`获取并整合远程修改，再通过`git push`将本地形成的commit同步到远程仓库。

在此基础上，GitHub、Gitee还提供Pull Request等协作方式。开发者可以先在自己的分支中完成修改，再提出合并请求，其他成员可以在正式合并之前查看代码差异、讨论修改内容并进行检查。对于公开项目，还可以通过fork获得自己的项目副本，在完成修改后通过Pull Request向原项目提交改进。

Git因此连接了本地版本管理、多人远程协作和开源项目。对于目前的学习阶段，课程实验、小组项目和实习任务已经会涉及代码修改、版本保存和多人协作；随着之后接触的项目规模扩大，Git所提供的版本历史、分支管理和远程协作方式也能够继续沿用。

综合来看，学习Git的意义主要体现在对代码变化和协作关系的管理上。commit和commit message使修改历史能够被记录和理解，分支和合并帮助组织多人或多任务开发，远程仓库则进一步把本地版本管理扩展到多人共享和开源协作。几种机制相互衔接，使代码从个人编写到多人整合都能够在较统一的版本体系中进行。

四、分支管理与冲突处理

以下以C表示首次修改后的基线提交，以F和A分别表示feature与main的独立提交，以M表示合并提交。

1. 创建feature分支

```
git rev-parse HEAD | tee ~/ics/lab0-evidence/base-commit.txt
git switch -c feature
git --no-pager branch
git --no-pager branch --show-current
```

创建feature时HEAD位于C。命令返回Switched to a new branch，分支列表中的星号标记feature。接下来，两个分支分别修改main.c的同一条printf，形成具有共同祖先的独立提交。

2. 分别修改两个分支

feature分支

在feature中将打印内容设为ICS Lab0: feature branch message.。保存源码后执行make和./main，输出与该分支内容一致，再清理构建产物并提交。

```
make
./main
make clean
git --no-pager diff -- main.c
git add main.c
git commit -m "feat: add the feature branch greeting"
git rev-parse HEAD | tee ~/ics/lab0-evidence/feature-commit.txt
git status
```

F为49e68ed。提交仅改变一条打印语句，状态显示On branch feature和working tree clean。

```

yonchn@YonChn:~/ics/lab0$ make clean
git --no-pager diff -- main.c
git add main.c
git commit -m "feat: add the feature branch greeting"
git rev-parse HEAD | tee ~/ics/lab0-evidence/feature-commit.txt
git status
git --no-pager branch --show-current
rm -f main.o main
diff --git a/main.c b/main.c
index 2fcf8c9..41d3731 100644
--- a/main.c
+++ b/main.c
@@ -2,6 +2,6 @@

int main(void)
{
-   printf("ICS Lab0: baseline message.\n");
+   printf("ICS Lab0: feature branch message.\n");
   return 0;
}
\ No newline at end of file
[feature 49e68ed] feat: add the feature branch greeting
1 file changed, 1 insertion(+), 1 deletion(-)
49e68ed6834f09d8147114dd38b673691a816ae2
On branch feature
nothing to commit, working tree clean
feature
yonchn@YonChn:~/ics/lab0$

```

图15 feature分支差异和提交

main分支

切回main后，文件恢复为基线内容，说明F没有直接改变main。我将同一位置改为ICS Lab0: main branch message.，保存并重新编译，验证该行输出后提交A。

在main分支编辑打印语句时，首次运行make和./main，终端仍输出ICS Lab0: baseline message.。保存文件后再次构建，输出变为ICS Lab0: main branch message.。

```

yonchn@YonChn:~/ics/lab0$ git --no-pager branch --show-current
main
make
./main
main
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
ICS Lab0: baseline message.
yonchn@YonChn:~/ics/lab0$ make
./main
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
ICS Lab0: main branch message.
yonchn@YonChn:~/ics/lab0$

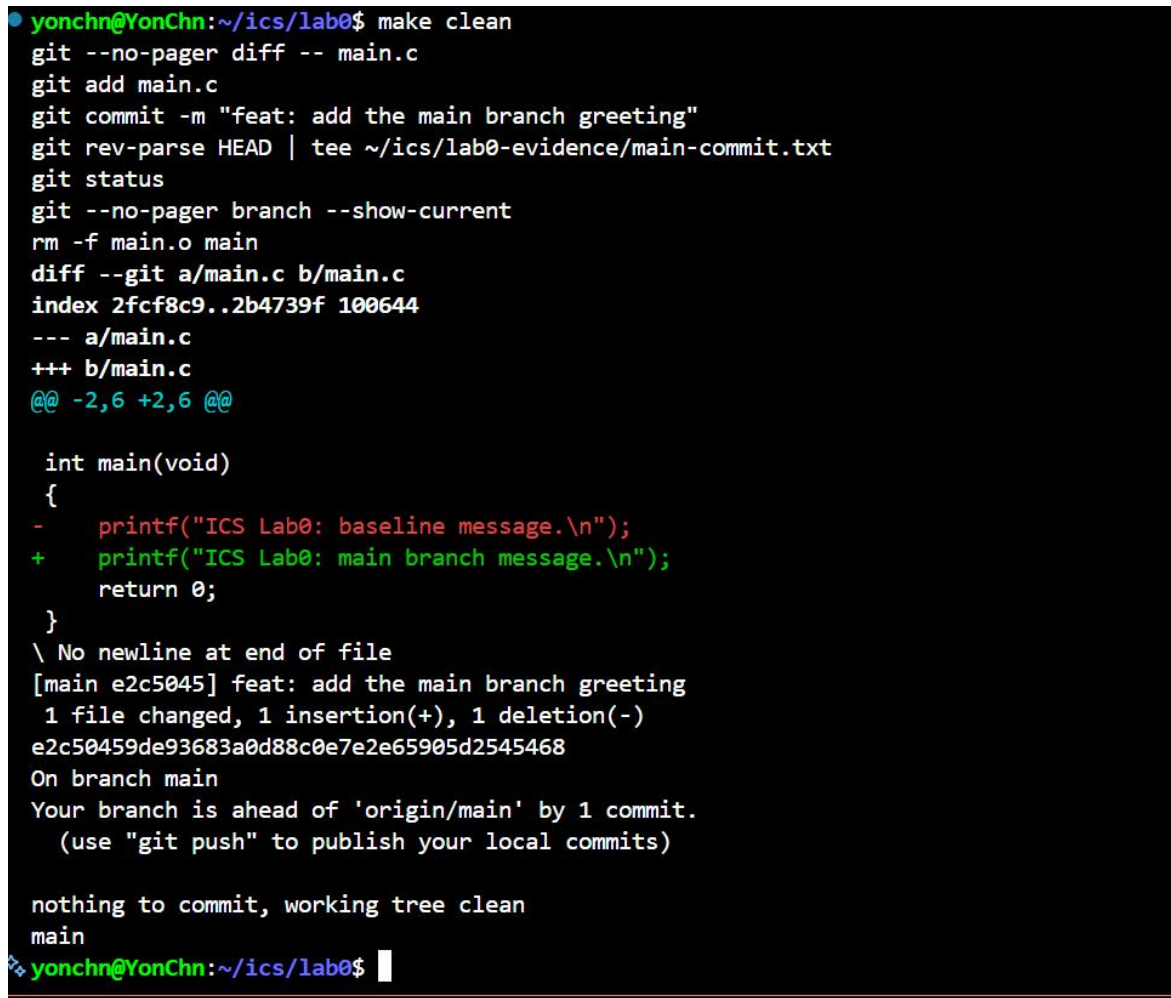
```

图16 保存前后两次构建的实际输出

编译器读取磁盘中的main.c，编辑器尚未保存的内容没有进入第一次构建。保存后，make重新编译并链接，程序输出随之改变；确认运行结果后，我提交了main分支的修改。下列两组make和./main分别对应保存前与保存后。

```
git switch main
make
./main
make
./main
make clean
git --no-pager diff -- main.c
git add main.c
git commit -m "feat: add the main branch greeting"
git rev-parse HEAD | tee ~/ics/lab0-evidence/main-commit.txt
git status
```

A为e2c5045。main工作区干净，领先origin/main 1个提交；feature仍指向F。



```
yonchn@YonChn:~/ics/lab0$ make clean
git --no-pager diff -- main.c
git add main.c
git commit -m "feat: add the main branch greeting"
git rev-parse HEAD | tee ~/ics/lab0-evidence/main-commit.txt
git status
git --no-pager branch --show-current
rm -f main.o main
diff --git a/main.c b/main.c
index 2fcf8c9..2b4739f 100644
--- a/main.c
+++ b/main.c
@@ -2,6 +2,6 @@

int main(void)
{
-   printf("ICS Lab0: baseline message.\n");
+   printf("ICS Lab0: main branch message.\n");
   return 0;
}
\ No newline at end of file
[main e2c5045] feat: add the main branch greeting
1 file changed, 1 insertion(+), 1 deletion(-)
e2c50459de93683a0d88c0e7e2e65905d2545468
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
main
yonchn@YonChn:~/ics/lab0$
```

图17 main分支差异和独立提交

3.比较合并前的版本

时点	main	feature	HEAD	源码中的打印内容
创建feature后	C	C	feature	ICS Lab0: baseline message.
feature提交后	C	F	feature	ICS Lab0: feature branch message.
main提交后	A	F	main	ICS Lab0: main branch message.

A和F的父提交都是C，两个分支从共同起点分别发展。

```
git --no-pager branch -v
git --no-pager show main:main.c
git --no-pager show feature:main.c
git merge-base main feature
git --no-pager log --graph --oneline --decorate --all
```

版本	提交缩写	打印内容
共同祖先C	2766614	ICS Lab0: baseline message.
main端点A	e2c5045	ICS Lab0: main branch message.
feature端点F	49e68ed	ICS Lab0: feature branch message.

git merge-base返回C的完整哈希，合并前日志显示A和F均从C分出。个人仓库初始提交981a9b1位于C之前。

4.触发并解决合并冲突

```
git --no-pager branch --show-current
git merge feature
git status
```

当前分支输出main，随后合并feature。终端显示CONFLICT (content): Merge conflict in main.c，git status将main.c列为both modified，并提示存在unmerged paths。HEAD一侧来自A，feature一侧来自F。

```

yonchn@YonChn:~/ics/lab0$ git --no-pager branch --show-current
git merge feature
main
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
yonchn@YonChn:~/ics/lab0$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   main.c

no changes added to commit (use "git add" and/or "git commit -a")
yonchn@YonChn:~/ics/lab0$

```

图18 main合并feature时出现内容冲突

```

<<<<<<< HEAD
    printf("ICS Lab0: main branch message.\n");
=====
    printf("ICS Lab0: feature branch message.\n");
>>>>>>> feature

```

两条分支都重写了共同祖先中的同一条printf，Git无法自动确定保留哪一条。随后我查看main.c和差异，并保存了冲突状态与双方内容。

```

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   main.c

no changes added to commit (use "git add" and/or "git commit -a")
#include <stdio.h>

int main(void)
{
<<<<<<< HEAD
    printf("ICS Lab0: main branch message.\n");
=====
    printf("ICS Lab0: feature branch message.\n");
>>>>>>> feature
    return 0;
yonchn@YonChn:~/ics/lab0$

```

图19 冲突状态与文件中的双方内容

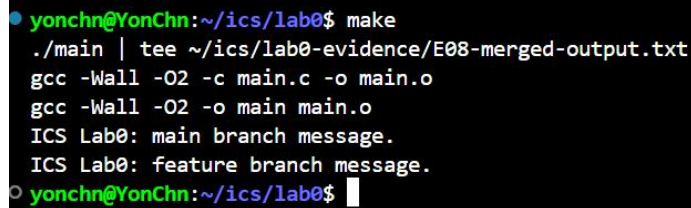
两个分支分别修改了共同祖先中的同一条打印语句，两项修改均有意保留。因此我删除Git插入的冲突标记，依次保留main和feature的输出。最终程序仍只有一个main函数，结束时返回0。

```
#include <stdio.h>

int main(void)
{
    printf("ICS Lab0: main branch message.\n");
    printf("ICS Lab0: feature branch message.\n");
    return 0;
}

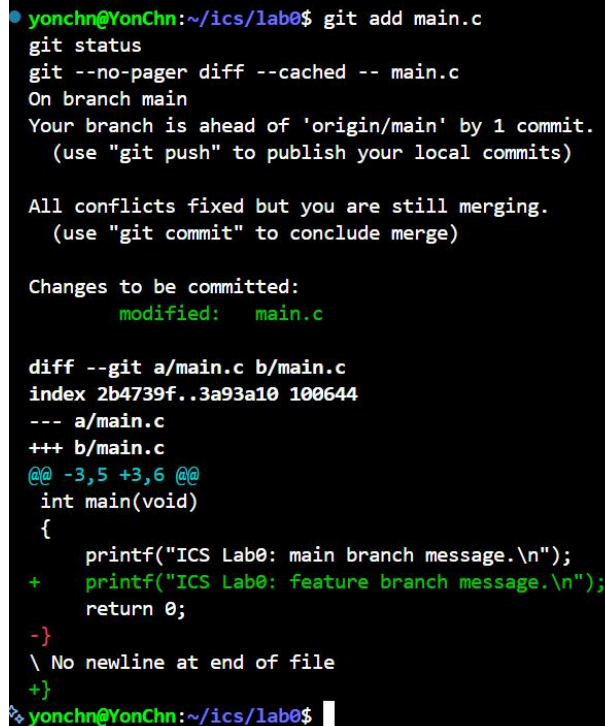
make
./main | tee ~/ics/lab0-evidence/E08-merged-output.txt
make clean
git --no-pager diff --check
git add main.c
git status
git --no-pager diff --cached -- main.c
```

构建成功，程序依次打印两条信息；diff --check没有报告问题。执行git add main.c将冲突解决结果暂存后，git status显示All conflicts fixed but you are still merging，说明冲突解决结果已进入暂存区，合并事务还需要提交结束。



```
yonchn@YonChn:~/ics/lab0$ make
./main | tee ~/ics/lab0-evidence/E08-merged-output.txt
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
ICS Lab0: main branch message.
ICS Lab0: feature branch message.
yonchn@YonChn:~/ics/lab0$
```

图20 冲突解决后的程序输出



```
yonchn@YonChn:~/ics/lab0$ git add main.c
git status
git --no-pager diff --cached -- main.c
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

All conflicts fixed but you are still merging.
(use "git commit" to conclude merge)

Changes to be committed:
  modified:   main.c

diff --git a/main.c b/main.c
index 2b4739f..3a93a10 100644
--- a/main.c
+++ b/main.c
@@ -3,5 +3,6 @@
 int main(void)
 {
     printf("ICS Lab0: main branch message.\n");
+    printf("ICS Lab0: feature branch message.\n");
     return 0;
-}
\ No newline at end of file
+}
yonchn@YonChn:~/ics/lab0$
```

图21 冲突解决结果已暂存

早期main.c缺少文件末尾换行，diff显示No newline at end of file。连续执行cat或git show时，后续文本会接在右花括号后，出现}`#include`。合并时补上文件末尾换行，暂存差异记录了这一变化。

5.建立合并提交

```
git commit -m "merge: preserve main and feature greetings"
git status
git --no-pager show -s --format="%H%n%P%n%s" HEAD
```

合并提交M为ec5e612，提交后工作区干净。输出的两个父哈希依次对应A和F，与main接收feature的合并方向一致。

```
yonchn@YonChn:~/ics/lab0$ git commit -m "merge: preserve main and feature greetings"
git status
git --no-pager show -s --format="%H%n%P%n%s" HEAD \
| tee ~/ics/lab0-evidence/E09-merge-parents.txt
[main ec5e612] merge: preserve main and feature greetings
On branch main
Your branch is ahead of 'origin/main' by 3 commits.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
ec5e6126fc6f56a14cd5c171e652910fb9f90f37
e2c50459de93683a0d88c0e7e2e65905d2545468 49e68ed6834f09d8147114dd38b673691a816ae2
merge: preserve main and feature greetings
yonchn@YonChn:~/ics/lab0$
```

图22 合并提交及两个父哈希

合并前记录的A和F与M的两个父节点一一对应，且A、F的共同起点为C。下式中的箭头由父提交指向子提交。

$$C \rightarrow A \rightarrow M \quad C \rightarrow F \rightarrow M$$

M同时记录两条开发线的历史来源。合并后main指向M，feature仍指向F。

6.推送与自动测试

```
git push origin main
git push -u origin feature
git fetch origin
git --no-pager log --graph --oneline --decorate --all
```

合并提交形成后，本地main已指向M；推送将远程main更新到M。feature推送后，远程对应分支指向F，本地feature设置为跟踪origin/feature。抓取后的图形日志中，HEAD指向main，main与origin/main对应M，origin/HEAD指向origin/main；feature与origin/feature对应F。

```
yonchn@YonChn:~/ics/lab0$ git push origin main
git push -u origin feature
git fetch origin
git --no-pager log --graph --oneline --decorate --all \
| tee ~/ics/lab0-evidence/E09-after-merge.txt
Enter passphrase for key '/home/yonchn/.ssh/id_ed25519':
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 20 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 878 bytes | 878.00 KiB/s, done.
Total 9 (delta 6), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (6/6), completed with 1 local object.
To github.com:KlineCY/ics-lab0.git
2766614..ec5e612 main -> main
Enter passphrase for key '/home/yonchn/.ssh/id_ed25519':
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'feature' on GitHub by visiting:
remote:   https://github.com/KlineCY/ics-lab0/pull/new/feature
remote:
To github.com:KlineCY/ics-lab0.git
* [new branch]      feature -> feature
Branch 'feature' set up to track remote branch 'feature' from 'origin'.
Enter passphrase for key '/home/yonchn/.ssh/id_ed25519':
*   ec5e612 (HEAD -> main, origin/main, origin/HEAD) merge: preserve main and feature greetings
| \
| * 49e68ed (origin/feature, feature) feat: add the feature branch greeting
* | e2c5045 feat: add the main branch greeting
| /
* 2766614 feat: print a personal lab greeting
* 981a9b1 Initial commit
yonchn@YonChn:~/ics/lab0$
```

图23 远程推送与合并后的提交图

两次推送分别触发GitHub Actions中的Hello World Modified测试。合并提交M推送到main对应Autograding Tests #3，feature分支提交F的推送对应#4；图24至图26记录的是#3，即合并版本的测试结果。运行环境标签为ubuntu-latest。该测试先清理并编译程序，检查./main的可执行权限，再将标准输出与模板中的Hello, world!及换行进行比较。输出与模板不同且前述检查通过时，该项测试通过。

记录项	实际结果
运行	Autograding Tests #3; 36522385691
对应分支与提交	main; ec5e6126fc6f56a14cd5c171e652910fb9f90f37
workflow结果	Success; 总耗时9秒
自动测试	Hello World Modified; pass
grade.json	score=100; max_score=100; passed=1; total=1
运行链接	https://github.com/KlineCY/ics-lab0/actions/runs/36522385691

复现项目	实际结果
目录	/home/yonchn/ics/lab0-check
本地HEAD	ec5e6126fc6f56a14cd5c171e652910fb9f90f37
远程refs/heads/main	ec5e6126fc6f56a14cd5c171e652910fb9f90f37
编译	gcc -Wall -O2编译并链接成功
运行输出	ICS Lab0: main branch message. ICS Lab0: feature branch message.

```

yonchn@YonChn:~/ics/lab0$ cd ~/ics/lab0
lab_remote_url=$(git remote get-url origin)
cd ~/ics
git clone "$lab_remote_url" lab0-check
cd ~/ics/lab0-check
pwd
git rev-parse HEAD
git ls-remote origin refs/heads/main
make
./main
Cloning into 'lab0-check'...
Enter passphrase for key '/home/yonchn/.ssh/id_ed25519':
remote: Enumerating objects: 20, done.
remote: Counting objects: 100% (20/20), done.
remote: Compressing objects: 100% (12/12), done.
remote: Total 20 (delta 7), reused 18 (delta 6), pack-reused 0 (from 0)
Receiving objects: 100% (20/20), 4.68 KiB | 4.68 MiB/s, done.
Resolving deltas: 100% (7/7), done.
/home/yonchn/ics/lab0-check
ec5e6126fc6f56a14cd5c171e652910fb9f90f37
Enter passphrase for key '/home/yonchn/.ssh/id_ed25519':
ec5e6126fc6f56a14cd5c171e652910fb9f90f37      refs/heads/main
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
ICS Lab0: main branch message.
ICS Lab0: feature branch message.
yonchn@YonChn:~/ics/lab0-check$

```

图27 独立克隆的哈希编译与运行结果

重新克隆后的HEAD与远程main的完整哈希一致，程序成功编译，并输出与原实验目录相同的两行信息。

五、实验结果与总结

本次实验建立了个人仓库，完成源码修改、暂存与提交，并使两个分支从共同基线独立发展。两条分支对同一打印语句的修改引发冲突；解决后，程序依次输出main与feature的信息。合并版本的自动测试得分为100/100，在新克隆目录中也成功构建并运行。

通过这些操作，我理解了暂存区对提交内容的选择、分支对独立修改的组织，以及合并提交对双方历史的记录。文件未保存时的旧输出也使我认识到，编辑器内容、磁盘源码和构建结果需要分别核对。

后续实验中，我将围绕明确任务组织提交，在可编译的阶段保留版本，合并后重新测试，并用提交哈希定位测试所对应的源码。